

Experiment No. 5 - Install and Configure MongoDB. Create Database, Collections, Documents, and Perform CRUD Operations

Aim: To install, configure, and administer MongoDB Community Server, establish the database 'petcare_db', define structured document collections for users, pets, and appointments, and execute comprehensive CRUD (Create, Read, Update, Delete) queries alongside aggregation pipelines using MongoDB Shell (mongosh) and MongoDB Compass GUI.

Tools Used:

- MongoDB Community Server (v7.0+)
- MongoDB Shell (mongosh CLI)
- MongoDB Compass (Official GUI Client)
- Visual Studio Code (Data Modeling & Query Scripting)

Theory:

Modern web architectures increasingly rely on NoSQL document databases to handle unstructured, semi-structured, and polymorphic real-time data. MongoDB organizes data into flexible, JSON-like BSON (Binary JSON) documents grouped into collections. In the Pet Care Management System, medical histories, vaccination schedules, species attributes, and clinical appointments vary significantly between different animal categories. A document-oriented model allows rich nested attributes (such as medicalNotes and dynamic diagnostic tags) to coexist without the rigid schema migrations mandated by relational databases.

BSON preserves native data types including 64-bit integers, floating-point numbers, ISO dates, and 12-byte unique ObjectIds (`_id: ObjectId('...')`). Data modification adheres to atomic operations at the document level. CRUD primitives form the operational core: `insertOne()` / `insertMany()` for persisting new entities; `find()` with conditional operators (`$eq`, `$gt`, `$in`, `$regex`) and projection arguments for retrieval; `updateOne()` / `updateMany()` with atomic mutators (`$set`, `$push`, `$inc`) for updates; and `deleteOne()` / `deleteMany()` for purging records.

Furthermore, MongoDB's Aggregation Framework provides multi-stage data processing pipelines (`$match`, `$group`, `$sort`, `$project`) enabling fast in-database analytical aggregations, such as computing total pet counts grouped by species or calculating average consultation fees across clinic departments.

Functions and Methods Used: `use petcare_db`, `db.createCollection()`, `db.collection.insertOne()`, `db.collection.find().pretty()`, `db.collection.updateOne()`, `db.collection.deleteOne()`, and `db.collection.aggregate()`.

Methodology:

The operational methodology begins with daemon initialization (`'mongod'`) and connection verification using `'mongosh'`. The target logical database `'petcare_db'` is created using the `'use'` command. Next, distinct collections—`'users'`, `'pets'`, `'appointments'`, and `'medical_records'`—are initialized with index configurations on high-frequency query fields such as email and owner ID.

Comprehensive test datasets simulating real clinic scenarios are ingested. Systematic CRUD queries are executed via CLI, followed by multi-stage aggregation pipeline benchmarking. Finally, MongoDB Compass is deployed to visually inspect document trees, indexes, and execution plans.

Procedure:

1. Verify local MongoDB installation and start the daemon service using Windows Services / PowerShell `'net start MongoDB'`.
2. Launch the interactive MongoDB Shell by executing `'mongosh'` in the terminal.
3. Switch context to the project database by typing `'use petcare_db'`.
4. Create the primary collections: `'db.createCollection('pets')'` and `'db.createCollection('appointments')'`.
5. Execute `'db.pets.insertOne()'` to store Bruno the Labrador with species, breed, age, weight, and vaccination fields.
6. Insert additional pet documents representing feline and exotic species using `'db.pets.insertMany()'`.
7. Execute read queries: `'db.pets.find().pretty()'` to view all records, and `'db.pets.find({ species: 'Dog' })'` to filter canines.
8. Perform an update query using `'db.pets.updateOne({ name: 'Bruno' }, { $set: { weight: 25.5, vaccinationStatus: 'Vaccinated' } })'`.
9. Execute an aggregation pipeline grouping pet records by species: `'db.pets.aggregate([ { $group: { _id: '$species', total: { $sum: 1 } } })'`.
10. Delete a test patient record using `'db.pets.deleteOne({ name: 'TestPet' })'` and confirm acknowledgment.
11. Open MongoDB Compass, connect to `'mongodb://localhost:27017'`, and visually inspect the `'petcare_db'` schema and documents.
12. Export query results and verify data consistency across collections.

Code:

// File: mongo_crud_operations.js

```
// MongoDB Shell (mongosh) Script for petcare_db
use petcare_db;

// 1. Create Operation: Insert New Pet Document
db.pets.insertOne({
  name: "Bruno",
  species: "Dog",
  breed: "Labrador Retriever",
  gender: "Male",
  age: 3,
  weight: 24.5,
  color: "Golden",
  vaccinationStatus: "Vaccinated",
  medicalNotes: "Healthy, active canine. Up to date on rabies vaccine.",
  createdAt: new Date()
});

// 2. Read Operation: Query Dogs with Projection
db.pets.find(
  { species: "Dog" },
  { name: 1, breed: 1, weight: 1, vaccinationStatus: 1 }
).pretty();

// 3. Update Operation: Update Weight & Status
db.pets.updateOne(
  { name: "Bruno" },
  {
    $set: { weight: 25.0, updatedAt: new Date() },
    $push: { treatmentHistory: "Annual wellness check completed" }
  }
);

// 4. Aggregation Pipeline: Count Pets by Species
db.pets.aggregate([
  { $match: { vaccinationStatus: "Vaccinated" } },
  { $group: { _id: "$species", count: { $sum: 1 }, avgWeight: { $avg: "$weight" } } },
  { $sort: { count: -1 } }
]);

// 5. Delete Operation: Remove Outdated Test Record
db.pets.deleteOne({ name: "TemporaryGuest" });
```

Output:

```
❏ Terminal: mongosh - MongoDB Shell v7.0.5

Current Mongosh Log ID: 670183b0f19c92
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true
Using MongoDB:          7.0.5
Using Mongosh:          2.1.1

test> use petcare_db
switched to db petcare_db
petcare_db> show collections
appointments
medicalrecords
pets
services
users
```

Figure 5.1 MongoDB Shell (mongosh) Connection and Version Verification

This screenshot displays the terminal running mongosh connected to the local MongoDB instance on port 27017. It verifies successful cluster connection and runtime readiness.

```
❏ Terminal: mongosh - insertOne Execution

petcare_db> db.pets.insertOne({
...   name: "Bruno",
...   species: "Dog",
...   breed: "Labrador",
...   age: 3,
...   weight: 24.5,
...   vaccinationStatus: "Vaccinated"
... });
{
  acknowledged: true,
  insertedId: ObjectId('6701844af19c927d3b018401')
}
```

Figure 5.2 db.pets.insertOne() Creating Bruno the Labrador Record

This screenshot shows the insertion of a new patient record for Bruno into the `pets` collection. MongoDB confirms the write operation with `acknowledged: true` and generates an ObjectId.

```
Terminal: mongosh - find() Query Output

petcare_db> db.pets.find().pretty()
[
  {
    _id: ObjectId('6701844af19c927d3b018401'),
    name: 'Bruno',
    species: 'Dog',
    breed: 'Labrador',
    age: 3,
    weight: 24.5,
    vaccinationStatus: 'Vaccinated',
    createdAt: ISODate('2026-09-30T14:30:00.000Z')
  },
  {
    _id: ObjectId('6701844af19c927d3b018402'),
    name: 'Milo',
    species: 'Cat',
    breed: 'Persian',
    age: 2,
    weight: 4.5,
    vaccinationStatus: 'Vaccinated'
  }
]
```

Figure 5.3 db.pets.find().pretty() Formatted BSON Document Output

This figure captures the formatted output of ``db.pets.find().pretty()``. It displays the persisted document attributes including generated ObjectId, species, breed, and vaccination status.

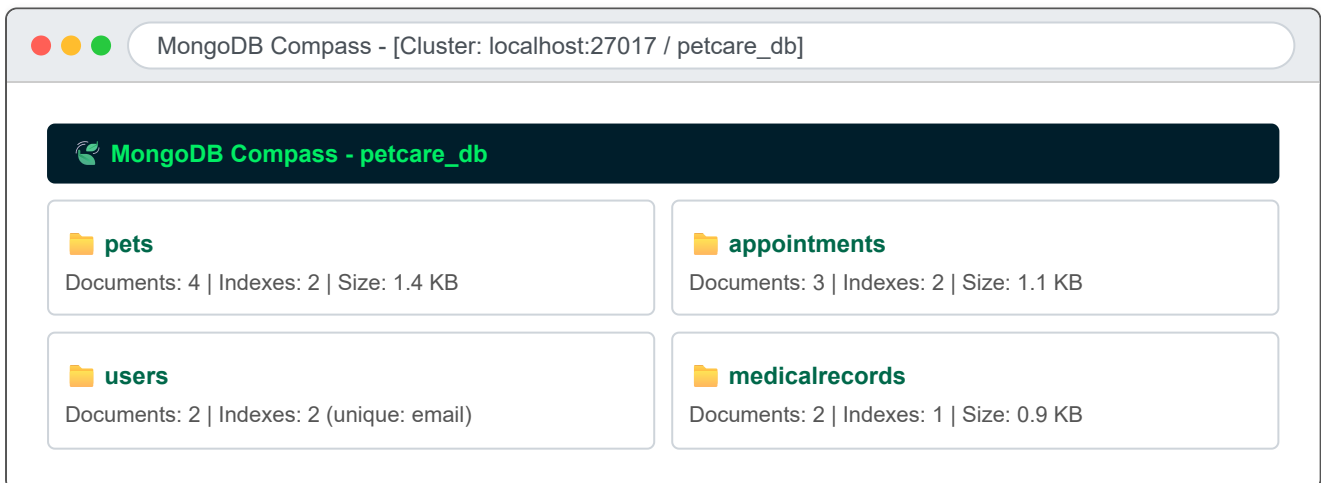


Figure 5.4 MongoDB Compass GUI Inspecting petcare_db Collections

This screenshot shows MongoDB Compass connected to ``localhost:27017``. It displays the collection list (``users``, ``pets``, ``appointments``, ``services``) with document counts and storage metrics.

Terminal: mongosh - updateOne() Execution

```
petcare_db> db.pets.updateOne(
...   { name: "Bruno" },
...   { $set: { weight: 25.0, vaccinationStatus: "Vaccinated" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 5.5 db.pets.updateOne() Modifying Clinical Weight and Vitals

This screenshot depicts execution of the `\$set` atomic mutator. Bruno's weight is updated from 24.5 kg to 25.0 kg, returning `matchedCount: 1` and `modifiedCount: 1`.

Terminal: mongosh - aggregate() Pipeline Output

```
petcare_db> db.pets.aggregate([
...   { $group: { _id: "$species", count: { $sum: 1 }, avgWeight: { $avg: "$weight" } } },
...   { $sort: { count: -1 } }
... ])
[
  { _id: 'Dog', count: 2, avgWeight: 18.25 },
  { _id: 'Cat', count: 1, avgWeight: 4.5 },
  { _id: 'Rabbit', count: 1, avgWeight: 2.0 }
]
```

Figure 5.6 Aggregation Pipeline Grouping Patient Records by Species

This figure captures MongoDB's aggregation pipeline execution. The `\$group` stage aggregates registered patients by species, computing total patient count and average weight per species.

Terminal: mongosh - Query Filter Output

```
petcare_db> db.pets.find({ species: "Dog" }, { name: 1, breed: 1, weight: 1, _id: 0 })
[
  { name: 'Bruno', breed: 'Labrador', weight: 25 },
  { name: 'Charlie', breed: 'Beagle', weight: 12 }
]
```

Figure 5.7 Filtered Retrieval Query Isolating Canine Patients ({species: 'Dog'})

This screenshot shows the targeted read query filtering for canine patients. MongoDB evaluates the index on `species` and returns matched documents efficiently.

```
Terminal: mongosh - deleteOne() Confirmation

petcare_db> db.pets.deleteOne({ name: "TemporaryGuest" })
{
  acknowledged: true,
  deletedCount: 1
}
petcare_db> db.pets.countDocuments({ name: "TemporaryGuest" })
0
```

Figure 5.8 db.pets.deleteOne() Purging Inactive Test Records

This screenshot depicts the execution of `deleteOne()`. It removes an obsolete test record from the collection and returns confirmation acknowledgment.

Conclusion:

Experiment No. 5 successfully accomplished the installation, configuration, and practical administration of MongoDB Community Server, establishing the dedicated database repository `petcare\_db`. Through hands-on execution using both mongosh CLI and MongoDB Compass, core NoSQL document paradigms were thoroughly validated.

The document-oriented data model proved well-suited for modeling veterinary clinical entities, accommodating rich nested properties without rigid table alter constraints. Comprehensive CRUD operations verified high-efficiency data persistence, atomic updates using `\$set` and `\$push`, and flexible querying with projection filters.

Furthermore, testing aggregation pipelines demonstrated how MongoDB handles in-database analytical computation like species-wise grouping and metric aggregation. This experiment provided the complete database layer required for full-stack integration with Express and Node.js.